BML Toolbox

Synchronization & Annotation System

Complete technical guide — Brain Modulation Laboratory

Brain Modulation Laboratory

Massachusetts General Hospital · University of Pittsburgh

April 2026

Covers: ROI tables · analog sync (Ripple/Zoom) · event sync (NeuroOmega, Psychtoolbox)
Old vs. new DP matching · consolidation · glossary · complete worked examples

Contents

1 Glossary of Key Terms **3**

2 System Overview and Data Flow **6**
2.1 The synchronization problem . . . 6
2.2 Complete data flow diagram . . . 6

3 Annotation Tables and the ROI Structure **7**
3.1 Annotation table schema . . . 7
3.2 ROI table: the sync state carrier . . . 8
3.3 Key annotation manipulation functions . . . 8

4 The Coordinate System **8**
4.1 Why two points? . . . 8

5 Building the Initial ROI Table **9**

6 Synchronizing Audio (Zoom) to Ripple/Trellis **10**
6.1 Approach A: Shared analog channel (`bml_sync_analog`) . . . 10
6.1.1 Inside `bml_sync_analog`: per-chunk pipeline . . . 10
6.2 Approach B: Digital triggers via audio-peak detection (`bml_sync_audio_event`) 10

7 Synchronizing NeuroOmega to Ripple/Trellis **11**

8 Synchronizing Task Laptop (Psychtoolbox) to Ripple **12**
8.1 The setup . . . 12
8.2 Step 1: Read the task log file . . . 12
8.3 Step 2: Get the same events from Ripple . . . 12
8.4 Step 3: Synchronize with `bml_sync_digital` . . . 13
8.5 Step 4: Apply sync to task events . . . 13
8.6 Step 4b: Exact-match fallback strategy . . . 13

9 Event Alignment: From Brute-Force to Dynamic Programming **15**
9.1 The problem with simple event matching . . . 15
9.2 The old approach (archived, pre-2025) . . . 15
9.3 The new approach (current) . . . 16
9.3.1 Summary of differences . . . 16
9.4 The dynamic programming algorithm (both versions) . . . 17
9.5 How `bml_sync_digital` uses the DP output . . . 17

10 Alignment Function Hierarchy **18**
10.1 Call graph . . . 18
10.2 The two dimensions . . . 18
10.3 Decision guide: which function to call? . . . 19

11 Event Alignment Internals: `bml_timealign_annot` **19**

12 Continuous Signal Alignment **20**
12.1 `bml_timealign` — cross-correlation . . . 20
12.2 `bml_timewarp` — linear warp optimization . . . 20

13 Consolidation: What It Is and Why It Matters **20**

13.1 The problem consolidation solves . . . 20
13.2 Level 1: per-file consolidation . . . 21
13.3 Level 2: contiguous file consolidation . . . 21
13.4 What the consolidated table looks like . . . 21

14 Sync Transfer Functions **22**
14.1 `bml_sync_transfer_trellis_filetype` — rescale sample indices . . . 22
14.2 `bml_sync_transfer_neuroomega_chantype` — re-anchor to channel internal clock . . . 23
14.3 Why not re-sync each file type separately? . . . 23
14.4 Putting it all together . . . 23

15 Complete End-to-End Example **24**

16 Common Pitfalls **25**

17 Diagnostic Plots: How to Enable and Interpret **26**
17.1 `bml_sync_match_events` — DP similarity plot . . . 26
17.2 `bml_sync_audio_event` / `bml_sync_neuroomega_event` — event alignment plot 27
17.3 `bml_sync_digital` — residual + alignment plot . . . 27
17.4 `bml_sync_consolidate` — pairwise residual heatmap . . . 28
17.5 Quick-reference: what to check first . . . 29

18 Best Practices and Lab Conventions **29**
18.1 Choose your sync method by signal quality . . . 30
18.2 Always validate sync quality . . . 30
18.3 Annotation tables as the lingua franca . . . 30
18.4 Save intermediate outputs . . . 30
18.5 The fallback-first principle . . . 31
18.6 BIDS event file conventions . . . 31
18.7 Chunking strategy . . . 31
18.8 Toward Python: design principles to carry forward . . . 31

A Mathematical Reference **32**

Glossary of Key Terms

This section defines every term used throughout the document. Read this first; refer back to it any time a term is unclear.

A standard MATLAB `table` with mandatory columns `id`, `starts`, `ends`, `duration`. The universal data structure in BML for representing any time-stamped event or segment — stimulus onsets, recording intervals, sync events, etc. Created and enforced by `bml_annot_table()`.

An annotation table extended with file-level columns: `folder`, `name`, `nSamples`, `Fs`, `filetype`, `chantype`, `s1`, `t1`, `s2`, `t2`. One row per file (or file chunk). This is the main carrier of synchronization state in the pipeline. Created by `bml_roi_table()`.

The **master** device is the one whose clock defines "ground truth" time for the entire session. In BML, this is always the **Ripple/Trellis** neural recorder. All other devices are **slaves**: their sample indices must be mapped to master-clock times. The master's `warpfactor` is always `1.000000` (it is its own reference).

Four numbers stored in every ROI table row that define a linear mapping from the file's sample indices to absolute master-clock times. `s1` and `s2` are two sample indices; `t1` and `t2` are the master-clock *midpoint times* of those samples. Together they encode both a time offset and a clock-rate correction. Before synchronization, `t1`/`t2` come from OS timestamps (unreliable). After synchronization, they are true master-clock values.

A sample at index i represents the signal averaged over a window of width $1/F_s$. Its "time" is the midpoint of that window. If a file starts at t_{start}, the midpoint of its first sample is $t_{\text{start}} + 0.5/F_s$. This $\pm 0.5/F_s$ correction appears everywhere in BML and is easy to overlook.

The ratio between the slave device's nominal sampling rate and its effective rate as measured against the master clock. `warpfactor > 1` means the slave clock ran *slow* (fewer samples per second than nominally); `warpfactor < 1` means it ran *fast*. Stored per file in the ROI table. Computed as $1/ws_1$ from the time-warp model. Typical values deviate from 1 by 1–10 ppm.

The phenomenon where two independent crystal oscillators nominally running at the same frequency (e.g., 30 000 Hz) actually run at slightly different rates. A drift of 10 ppm means that after one hour, the two clocks differ by 36 ms. Drift is corrected by the `ws1` parameter in `bml_timewarp`, or by the `warpfactor` in event-based sync.

A time window (defined in master time) used to subdivide a recording session for synchronization. Each chunk is synced independently, producing one row per (file × chunk) in the preliminary sync ROI. Chunking serves two purposes: (1) it limits memory usage by loading only part of each file at once, and (2) it provides multiple independent alignment estimates for the same file, which can be validated against each other during consolidation. Chunks are defined by `bml_chunk_sessions()`.

The process of merging multiple sync estimates (from different chunks of the same file, or from adjacent files of the same type) into a single, validated linear mapping per file. Think of it as fitting a "best line" through all the individual sync results and checking that they agree. If they do (residuals < 1 ms), the line is accepted. If they don't, an error is raised. Performed by `bml_sync_consolidate()`. See Section 13 for details.

Two or more files from the same device that cover consecutive time intervals with no gap (e.g., a Zoom recorder that auto-splits into hourly files). Contiguous files share the same physical clock and can be consolidated into a single linear mapping. The boundary between them is set at the midpoint of their adjacent anchor times.

The physical signal used to establish synchronization. For analog sync, this is a channel present in both the master and slave recordings (e.g., a click track on Trellis `ainp1` and Zoom audio). For event sync, this is a set of discrete events simultaneously visible on both devices (e.g., TTL pulses on Trellis digital input and NeuroOmega digital input).

The coarse synchronization stage in `bml_sync_analog`. Both signals are processed through the analytic signal envelope (absolute value of Hilbert transform, downsampled to 100 Hz) before cross-correlation. This produces a smooth, low-frequency representation that is robust to large time offsets (± 300 s) and polarity inversions, but has limited timing resolution ($\sim$10 ms).

The fine synchronization stage in `bml_sync_analog`. Both signals are low-pass filtered (4th-order zero-phase Butterworth, cutoff $\leq$4000 Hz) and cross-correlated within a narrow scan window (± 1 s). This achieves sub-millisecond timing resolution but requires the coarse alignment to already be within the scan window.

Low-level function that finds the time offset between two continuous signals by cross-correlation. Used inside `bml_timewarp` (and by extension `bml_sync_analog`) for both the envelope and LPF stages. Returns `slave_delta_t` and `max_corr`.

bml_timewarp

Low-level function that finds both the time offset *and* clock drift between two continuous signals by first calling `bml_timealign` and then running a non-linear optimization

(Nelder-Mead simplex) to maximize signal correlation under a linear time-warp model. Returns new sync coordinates `(s1, t1, s2, t2, wt0, ws1)`.

Event-based alignment function. Given two sets of discrete events (as annotation tables), finds the time shift (and optionally warp) that minimizes the sum of mismatches between slave events and their nearest master counterparts. Uses brute-force scan + Nelder-Mead refinement. Used by audio and NeuroOmega sync.

Event-*pairing* function using dynamic programming. Unlike `bml_timealign_annot` (which assumes most events are paired), this function handles sequences where events may be missing on either side. It finds the optimal subset of paired events that maximizes a weighted similarity score. Used by `bml_sync_digital` for Psychtoolbox/task laptop sync. See Section 9 for a detailed comparison with the older approach.

The Brain Imaging Data Structure tab-separated format for event logs. Columns: `onset` (seconds from session start) and `duration`. BML reads these via `bml_annot_read_tsv()`, which auto-renames `onset` → `starts` and computes `ends = starts + duration`.

The master neural recorder used in the lab. Trellis is the software; Ripple is the hardware brand. The Ripple system records ECoG, LFP, and analog/digital auxiliary channels. Its file format (`.nev` for events, `.ns*` for continuous data) is read by the NPMK library. In BML, `filetype = 'trellis'`.

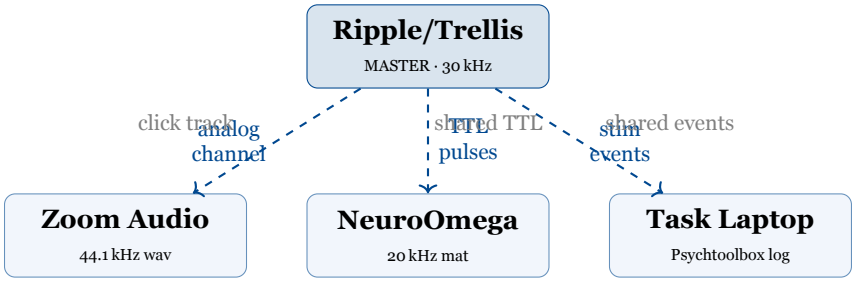
A microelectrode recording system (Alpha Omega) used for intraoperative MER recordings. Records neural signals and digital events in proprietary `.mat` files. Synchronized to Trellis via shared digital trigger pulses using `bml_sync_neuroomega_event()`.

A portable audio recorder (Zoom H-series) used to capture speech and a sync click track. Synchronized to Trellis either via a shared analog channel (`bml_sync_analog`) or via peak detection (`bml_sync_audio_event`).

System Overview and Data Flow

The synchronization problem

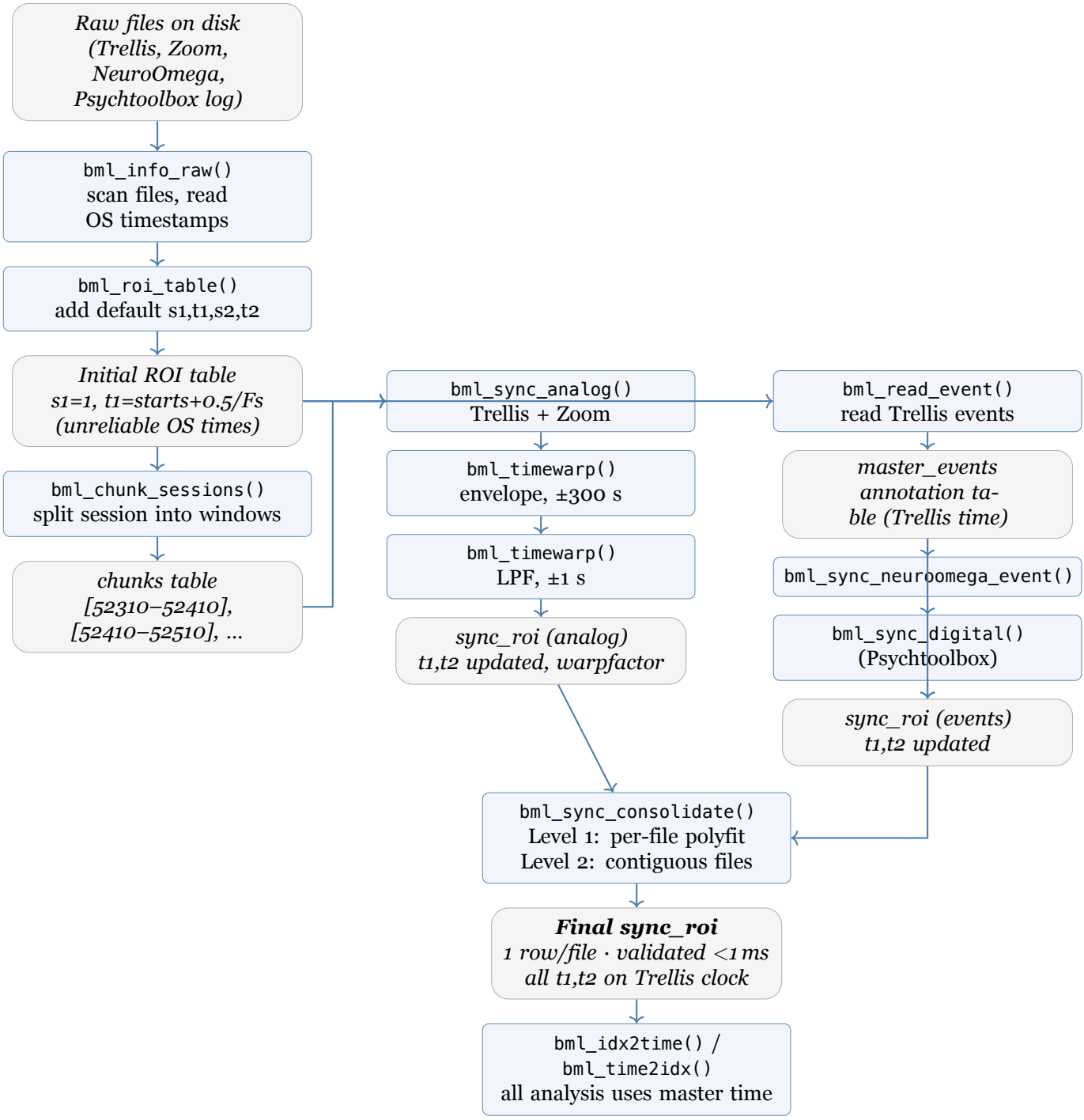
A BML recording session involves up to four independent devices, each with its own clock:



Each slave device's clock differs from the master by an **offset** (δt, seconds to minutes) and **drift** (ξ, 1–10 ppm). Both are corrected by a linear model: $t_{\text{master}} = \delta t + \xi \cdot t_{\text{slave}}$.

Complete data flow diagram

The diagram below shows every step from raw files to synchronized data. Boxes are functions; rounded boxes are data structures; arrows show data flow.



Annotation Tables and the ROI Structure

Annotation table schema

Every annotation table is created by `bml_annot_table()` which enforces the schema:

Column	Type	Description
`id`	integer	Auto-assigned by sorting on `starts`. Not stable across calls.
`starts`	double (s)	Start time in seconds from midnight
`ends`	double (s)	End time in seconds from midnight
`duration`	double (s)	Always `ends - starts`; never set manually
user cols	any	Preserved after the core four

ROI table: the sync state carrier

Column	Type	Description
`folder, name`	string	File location
`nSamples, Fs`	num	File length and nominal sample rate
`filetype`	string	`'trellis'`, `'zoom'`, `'neuroomega'`, ...
`chantype`	string	`'analog'`, `'digital'`, `'all'`
`s1`	integer	First reference sample index
`t1`	double (s)	Master-clock midpoint time of sample `s1`
`s2`	integer	Second reference sample index
`t2`	double (s)	Master-clock midpoint time of sample `s2`
`warpfactor`	double	$1/ws_1$; `1` = no drift
`chunk_id`	integer	Which chunk produced this row
`sync_type`	string	`'master'` or `'slave'`

Key annotation manipulation functions

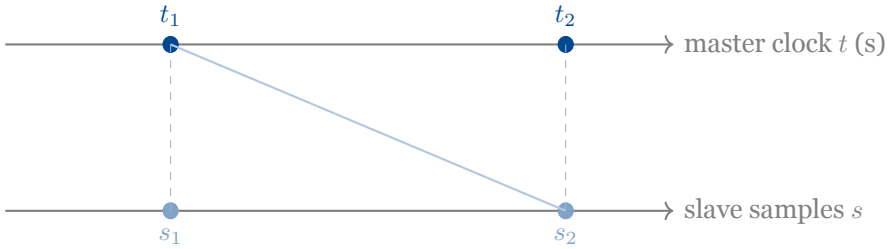
Function	What it does
`bml_annot_table(x)`	Constructor. Enforces schema, sorts, assigns `id`, computes `duration`.
`bml_annot_overlap`	Returns all row pairs with overlapping intervals.
`bml_annot_intersect`	Set intersection: intervals where x and y overlap, merged columns.
`bml_annot_union`	Concatenate and consolidate touching/overlapping intervals.
`bml_annot_difference`	Subtract y's time regions from x.
`bml_annot_filter`	Keep only rows of `a` that intersect filter `f`.
`bml_annot_consolidate`	Merge contiguous/overlapping rows (run-length encode).
`bml_annot_extend`	Shift `starts -= ext1`, `ends += ext2` (temporal buffers).
`bml_annot_read_tsv`	Read BIDS TSV event logs; renames `onset`→`starts`.
`bml_roi_confluence`	Set midpoint junction times between adjacent file chunks.

The Coordinate System

Why two points?

A single time offset δt corrects when a recording started but not how fast its clock ran. Two reference points (s_1, t_1) and (s_2, t_2) encode *both*: the slope of the line through them gives the

effective rate; the intercept gives the offset.



unique linear map: slope $= 1/F_{s,\text{eff}}$, intercept = time offset

$$F_{s,\text{eff}} = (s_2 - s_1)/(t_2 - t_1) \qquad t(i) = i/F_{s,\text{eff}} - 0.5/F_{s,\text{eff}} + (s_2 t_1 - t_2 s_1)/(s_2 - s_1)$$

$$i = \text{round}((t_2 s_1 - s_2 t_1 + (s_2 - s_1)\, t) \,/\, (t_2 - t_1))$$

Before sync: $F_{s,\text{eff}} = F_s$ (nominal), intercept from OS timestamp (seconds off).
After sync: t_1, t_2 are true master-clock values; $F_{s,\text{eff}}$ may differ slightly from F_s.

Building the Initial ROI Table

```
% bml_info_raw scans the directory and reads OS file metadata
cfg = [];
cfg.folder   = '/data/patient_001/session_01';
cfg.filetype = {'trellis','zoom','neuroomega'};
info = bml_info_raw(cfg);
% info: starts = file_modification_time - nSamples/Fs
%       ends   = file_modification_time  (from OS, unreliable ±seconds)

% bml_roi_table adds default sync coordinates
roi = bml_roi_table(info);
%   s1 = 1
%   t1 = starts + 0.5/Fs   (midpoint of first sample -- from OS time)
%   s2 = nSamples
%   t2 = ends   - 0.5/Fs   (midpoint of last sample  -- from OS time)
% NOTE: t1 and t2 are WRONG until synchronization is done.
```

Listing 1: Step 1: scan files and build the initial ROI

```
session = bml_annot_table(table(52310, 52610, 'VariableNames', {'starts','ends'})
    );
session.session_id = 1;

chunks = bml_chunk_sessions(session, 3);   % split into 3 equal chunks
% OR: bml_chunk_sessions(session, 52450)   % split at a specific time
% OR: bml_chunk_sessions(session, [], 60)  % split into 60-second pieces

% chunks is an annotation table with columns:
%   id, starts, ends, duration, session_id, session_part
```

Listing 2: Step 2: define session chunks

Synchronizing Audio (Zoom) to Ripple/Trellis

Approach A: Shared analog channel (`bml_sync_analog`)

Both Trellis and Zoom record the same click track on an analog channel. `bml_sync_analog` finds the time shift and drift by cross-correlating these signals.

```
sync_channels = table( ...
  {'trellis'; 'zoom'}, {'ainp1'; 'Ch1'}, {'analog'; 'analog'}, ...
  'VariableNames', {'filetype','channel','chantype'});
sync_channels.threshold = [NaN; NaN];  % NaN = no saturation zeroing

cfg_analog = [];
cfg_analog.roi             = roi(ismember(roi.filetype,{'trellis','zoom'}), :);
cfg_analog.chunks          = chunks;
cfg_analog.master_filetype = 'trellis';
cfg_analog.sync_channels   = sync_channels;
cfg_analog.timewarp        = true;
cfg_analog.env_scan        = 300;    % coarse: ±300 s envelope xcorr
cfg_analog.lpf_scan        = 1;     % fine:   ±1 s LPF xcorr
cfg_analog.lpf_max_freq    = 4000;
cfg_analog.chunk_extend    = 5;     % load 5 s extra each side
sync_roi_analog = bml_sync_analog(cfg_analog);
```

Inside `bml_sync_analog`: per-chunk pipeline

For each chunk k, for each slave filetype:

1. **Load** master sync channel (`ainp1`) and slave sync channel (`Ch1`) as `FT_DATATYPE_RAW`.
2. **Coarse alignment** — envelope warp (lines 322–327):

```
cfg.method = 'envelope'; cfg.env_freq = 100; cfg.scan = 300;
cfg.penalty_wt0_min = 1e-3; cfg.penalty_ws1 = 1e-3;
wc_env = bml_timewarp(cfg, master, slave);
slave.time{1} = bml_idx2time(wc_env, 1:length(slave.time{1}));
```

3. **Fine alignment** — LPF warp (lines 338–348):

```
cfg.method = 'low-pass-filter'; cfg.scan = 1;
cfg.lpf_freq = min([master.fsample, slave.fsample, 4000]);
cfg.penalty_wt0_min = 1e-6; cfg.penalty_ws1 = 1e-4;
wc_lpf = bml_timewarp(cfg, master, slave);
slave.time{1} = bml_idx2time(wc_lpf, 1:length(slave.time{1}));
```

4. **Write output row** (lines 352–367):

```
row.t1         = bml_idx2time(wc_lpf, slave_map.raw1); % true master-clock
    times
row.t2         = bml_idx2time(wc_lpf, slave_map.raw2);
row.starts     = row.t1 - 0.5 ./ row.Fs;
row.ends       = row.t2 + 0.5 ./ row.Fs;
row.warpfactor = wc_env.ws1 * wc_lpf.ws1;  % combined warp from both stages
```

Approach B: Digital triggers via audio-peak detection (`bml_sync_audio_event`)

Approach A requires both devices to record the same *continuous* analog waveform. Approach B does not: Ripple/Trellis logs the TTL click pulses as **digital events** (integers in its

event channel), and the Zoom recorder captures the same pulses acoustically as amplitude peaks. `bml_sync_audio_event` detects those peaks with `findpeaks` and matches them to the Trellis digital event log. This is functionally the *digital* counterpart of Approach A, and is labelled `sync_channel='digital'` in the output table.

When to use each approach. Use Approach A (analog) when both devices record the sync channel as a waveform: it is more robust because it exploits the full signal shape, not just peak positions. Use Approach B (digital peaks) when the Zoom recording only carries a click track and there is no shared analog channel on Trellis — the only link is the TTL events on the Ripple digital input and the acoustic peaks in the Zoom audio.

```
% master_events come from Ripple's DIGITAL event channel (TTL pulses)
master_events = bml_read_event(trellis_roi);
master_events = bml_event2annot([], master_events);
% Keep only click/sync events (e.g. value==1)
master_events = master_events(master_events.value == 1, :);

cfg = [];
cfg.roi           = roi(strcmp(roi.filetype,'zoom'), :);
cfg.master_events = master_events;   % <-- digital events from Ripple
cfg.scan          = 100;    cfg.scan_step = 0.1;
cfg.min_rph       = 0.5;    % peaks must be > 50% of file max amplitude
cfg.min_ipi       = 0.05;   % peaks must be > 50 ms apart
cfg.timewarp      = false;
cfg.diagnostic_plot = true;
sync_roi_zoom = bml_sync_audio_event(cfg);
% sync_roi_zoom.sync_channel == 'digital'
```

Per file: `findpeaks(abs(audio))` → build slave event table → `bml_timealign_annot` → apply shift/warp to `t1,t2`. Contiguous files averaged together.

Synchronizing NeuroOmega to Ripple/Trellis

NeuroOmega shares TTL pulse events with Trellis via a digital input. Both devices log the same physical pulses in their own clocks.

```
master_events = bml_read_event(trellis_roi);
master_events = bml_event2annot([], master_events);
master_events = master_events(strcmp(master_events.type,'STIM_ON'), :);

cfg = [];
cfg.roi           = roi(strcmp(roi.filetype,'neuroomega'), :);
cfg.master_events = master_events;
cfg.scan          = 100;   cfg.scan_step = 0.1;
cfg.coarsetol     = 100;   % restrict master events to ±100 s window
cfg.timewarp      = false; % true for sessions > 30 min
cfg.timetol       = 1e-6;
cfg.diagnostic_plot = true;
sync_roi_no = bml_sync_neuroomega_event(cfg);
```

Per file:

1. `bml_read_event(roi(i,:))` — read NeuroOmega events
2. `bml_event2annot()` — convert to annotation table

3. `bml_annot_filter(master, bml_annot_extend(roi(i,:), coarsetol))` — narrow master to ±100 s window
4. `bml_timealign_annot(cfg, master_events, slave_events)` — find δt (and optionally ξ)
5. Update `t1,t2` via: `roi.t1 = (roi.t1 - midpt) .* warpfactor + midpt + slave_dt`

Synchronizing Task Laptop (Psychtoolbox) to Ripple

The setup

A task laptop runs Psychtoolbox (or PsychoPy) and logs every stimulus onset, response, and event with its own system clock. The same task sends TTL pulses to both the Ripple digital input and the task laptop (or the laptop time-stamps events that coincide with Ripple-logged pulses). The goal is to map task laptop event times onto the Trellis master clock.

Step 1: Read the task log file

Task logs are typically saved as TSV or CSV files following the BIDS convention:

```
% BIDS TSV format: columns 'onset' and 'duration' (both in seconds from task
    start)
% bml_annot_read_tsv auto-renames 'onset' -> 'starts', computes 'ends'
ptb_events = bml_annot_read_tsv('sub-DM1001_ses-intraop_task-speech_events.tsv');
% ptb_events: id, starts, ends, duration, trial_type, response_time, ...
% starts = seconds from task start (laptop clock, NOT master clock)
```

Listing 3: Reading a Psychtoolbox / BIDS event log

For non-BIDS formats, build the annotation table manually:

```
raw = readtable('psychtoolbox_log.csv');
% raw has columns: onset_time, event_type, trial_id

ptb_events = bml_annot_table(table( ...
  raw.onset_time, ...          % starts (laptop clock seconds)
  raw.onset_time, ...          % ends (instantaneous events)
  'VariableNames', {'starts','ends'}));
ptb_events.type  = raw.event_type;
ptb_events.value = ones(height(ptb_events), 1);
```

Step 2: Get the same events from Ripple

The Ripple digital input recorded the same TTL pulses that the laptop sent (or received). Extract them:

```
master_events = bml_read_event(trellis_roi);
master_events = bml_event2annot([], master_events);
% Keep only the event type corresponding to task triggers
master_events = master_events(master_events.value == 1, :);
% master_events.starts = times in TRELLIS clock [seconds]
```

Step 3: Synchronize with `bml_sync_digital`

```
cfg = [];
cfg.timetol        = 1e-3;   % 1 ms tolerance
cfg.sim_threshold  = 0.9;    % minimum similarity to accept a match
cfg.diagnostic_plot = true;
cfg.plot_title     = 'Psychtoolbox to Trellis';

% Weight configuration for event matching
cfg.timetol           = 1e-3;   % timing tolerance for similarity kernel
cfg.weight_time_pre   = 1/4;    % weight for pre-event interval match
cfg.weight_time_post  = 1/4;    % weight for post-event interval match
cfg.weight_value_pre  = 1/4;    % weight for pre-event value match
cfg.weight_value_post = 1/4;    % weight for post-event value match
cfg.weight_onset      = 0;      % set > 0 to prefer temporally close matches

[sync_roi_ptb, hF] = bml_sync_digital(cfg, master_events, ptb_events);
% sync_roi_ptb columns:
%   starts, ends   -- time window of matched events (in master time)
%   delta_t        -- shift to add to laptop times to get master times
%   warpfactor     -- clock rate correction (1 = no drift)
%   mean_sim       -- mean similarity score of matched event pairs
%   s1, t1, s2, t2 -- sync coordinates (from sample column if available)
```

Listing 4: Synchronizing Psychtoolbox events to Trellis

Step 4: Apply sync to task events

After synchronization, convert all laptop event times to master clock:

```
% Apply the linear transformation from sync_roi_ptb
delta_t    = sync_roi_ptb.delta_t(1);
warpfactor = sync_roi_ptb.warpfactor(1);
tbar       = mean(ptb_events.starts);   % pivot for warping

ptb_events_synced = ptb_events;
ptb_events_synced.starts = (ptb_events.starts - tbar) .* warpfactor + tbar +
    delta_t;
ptb_events_synced.ends   = (ptb_events.ends   - tbar) .* warpfactor + tbar +
    delta_t;
% Now ptb_events_synced.starts are in TRELLIS (master) clock seconds
```

Step 4b: Exact-match fallback strategy

`bml_sync_digital` returns a single linear model. A more precise strategy, used in production scripts, exploits the DP matching directly:

- For task events that **matched a Ripple event** in the DP: substitute the exact Ripple timestamp. No interpolation error.
- For task events in **gaps** (no matching Ripple event — task events the Ripple didn't log): use the linear warp as a fallback.

This hybrid approach gives sub-millisecond accuracy for most events while gracefully handling trials where the Ripple event was not recorded.

```
% SYNCHRONIZING EVENT FILES TO RIPPLE EVENTS
```

```
% Will inherit exact timing from Ripple. Looping through events_files.
events_files.delta_t(:)    = nan;
events_files.warpfactor(:) = nan;
events_files.n(:)          = nan;

for i = 1:height(events_files)

  % Skip if no matching Ripple (NEV) events exist for this file
  % (prevents matching pre-op events to intraop NEV files)
  nev_events_files = bml_annot_filter(roi_nev, events_files(i,:));
  if isempty(nev_events_files); continue; end

  task_events = bml_annot_read_tsv( ...
    fullfile(events_files.path{i}, events_files.filename{i}));

  % DP matching: find which task events correspond to which master events
  cfg = [];
  cfg.timetol        = 0.003;   % 3 ms timing tolerance for similarity kernel
  cfg.weight_onset   = 0.1;     % small onset weight to prefer temporally close
      matches
  cfg.diagnostic_plot = true;
  [idxs_master, idxs_task, mean_sim, simv] = ...
    bml_sync_match_events(cfg, master_events, task_events);

  % Find the longest contiguous block of matched pairs
  % (guards against spurious matches at session boundaries)
  disconts_idx       = [1, find(diff(idxs_master) > 3), length(idxs_master)];
  [~, cont_idx]      = max(diff(disconts_idx));
  start_idx          = disconts_idx(cont_idx) + 1;
  end_idx            = disconts_idx(cont_idx + 1) - 1;

  % Fit linear warp as FALLBACK (for events without a direct Ripple match)
  tbar = mean(task_events.starts(idxs_task), 'omitnan');
  p    = polyfit( ...
    task_events.starts(idxs_task(start_idx:end_idx))   - tbar, ...
    master_events.starts(idxs_master(start_idx:end_idx)) - tbar, 1);

  events_files.warpfactor(i) = p(1);
  events_files.delta_t(i)    = p(2);
  events_files.n(i)          = height(task_events(idxs_task, :));

  % Compute fallback times for ALL task events via the linear model
  task_events.starts_corrected = ...
    (task_events.starts - tbar) .* p(1) + tbar + p(2);

  % PRIMARY: substitute exact Ripple time for matched events
  task_events.starts = bml_map( ...
    1:height(task_events), ...
    idxs_task, master_events.starts(idxs_master), nan);

  % FALLBACK: fill gaps with linear-warp times
  task_events.starts(isnan(task_events.starts)) = ...
    task_events.starts_corrected(isnan(task_events.starts));

  % Save synchronized event file
  bml_annot_write_tsv(task_events, ...
    fullfile(PATH_ANNOT, strrep(events_files.filename{i}, '.tsv', '-sync.tsv')));

```

```
  fprintf('sync %s  delta_t=%.4f  mean_sim=%.3f  warpfactor=%.7f\n', ...
    events_files.filename{i}, p(2), mean_sim, p(1));
end
```

Listing 5: Fallback strategy: exact Ripple times where available, linear interpolation elsewhere

Why two passes? The linear warp corrects for clock drift globally. But Ripple's event timestamps are the *ground truth*: they were recorded at hardware precision (30 kHz sample clock). Where a direct match exists, using the Ripple timestamp is always more accurate than any interpolated value. The fallback linear model fills only the gaps.

Choosing the longest contiguous block. The DP matching may produce a few spurious matches at the very start or end of the session (where the event sequences don't overlap well). Taking the longest contiguous block — defined by no gap of more than 3 unmatched master events — avoids these boundary artefacts in the linear fit.

Event Alignment: From Brute-Force to Dynamic Programming

The problem with simple event matching

Both `bml_sync_audio_event` and `bml_sync_neuroomega_event` use `bml_timealign_annot`, which works well when:

- Most slave events have a corresponding master event (few missing events)
- The initial OS-timestamp coarse alignment is roughly correct

But for Psychtoolbox/task events, **neither assumption may hold**:

- The laptop clock can be hours off from the master
- Some trial types may be absent on one side
- The event sequence may contain events of many different values

`bml_sync_match_events` solves this with dynamic programming.

The old approach (archived, pre-2025)

The archived version (`bml_sync_match_events_archivePLB20250715.m`) used a **4-feature** similarity function:

```
% Feature vector for event i (4 features):
% [ interval_before_i, interval_after_i, value[i], value[i+1] ]
x1 = [events1_delta_starts(1:end-1), ...   % pre-interval
      events1_delta_starts(2:end),   ...   % post-interval
      events1.value(1:end-2),        ...   % pre value
      events1.value(2:end-1)];            % post value

% Similarity (old): no onset term, fixed equal weights
similarity_fun = @(x1,x2) ...
  wdt1 .* (1./(1+((x1(:,1)-x2(:,1))./timetol).^2)) + ...  % pre-interval
  wdt2 .* (1./(1+((x1(:,2)-x2(:,2))./timetol).^2)) + ...  % post-interval
```

```
  wv1  .* (x1(:,3)==x2(:,3)) + ...                    % pre-value
  wv2  .* (x1(:,4)==x2(:,4));                         % post-value
```

Listing 6: Old 4-feature similarity (archived)

Problem: without an onset term, the algorithm had no preference for matches that are close in *absolute* time. Two events with identical inter-event intervals but 1 hour apart in the session were considered equally good candidates. In sessions with repetitive, regular event patterns, this could lead to mismatches.

The new approach (current)

The current version adds a **5th feature**: the absolute onset time of each event. This breaks ties by preferring pairs that are close in the session:

```
% Feature vector for event i (5 features):
% [ pre-interval, post-interval, pre-value, post-value, ONSET TIME ]
x1 = [events1_delta_starts(1:end-1), ...
      events1_delta_starts(2:end),   ...
      events1.value(1:end-2),        ...
      events1.value(2:end-1),        ...
      events1.starts(1:end-2)];      % <-- NEW: absolute onset time

% Similarity (new): onset term with configurable weight
similarity_fun = @(x1,x2) ...
  wdt1 .* (1./(1+((x1(:,1)-x2(:,1))./timetol).^2))   + ...  % pre-interval
  wdt2 .* (1./(1+((x1(:,2)-x2(:,2))./timetol).^2))   + ...  % post-interval
  wv1  .* (x1(:,3)==x2(:,3))                          + ...  % pre-value
  wv2  .* (x1(:,4)==x2(:,4))                          + ...  % post-value
  wo   .* (1./(1+((x1(:,5)-x2(:,5))./onsettol).^2));       % <-- onset time

% All weights are normalized to sum to 1
total = wdt1 + wdt2 + wv1 + wv2 + wo;
wdt1 = wdt1/total;  % etc.
```

Listing 7: New 5-feature similarity (current)

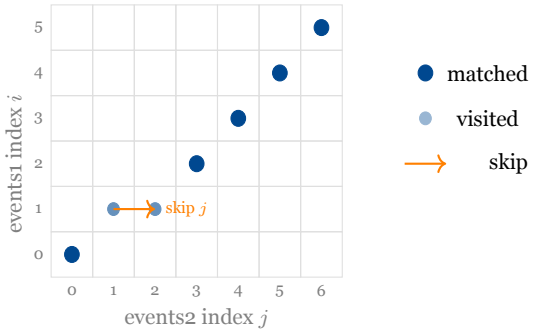
Summary of differences

Aspect	Old (archived)	New (current)
Features per event	4	5
Onset term	Absent	Present (`weight_onset`, default 0)
Weight params	`weight_time_pre/post`, `weight_value_pre/post`	Same + `weight_onset` + `onsettol`
Normalization	Fixed 1/4 each	Sum-normalized
Robustness	Can confuse repetitive patterns	Onset weight breaks ties
Backwards compat	—	`weight_onset=0` replicates old behavior

Setting `cfg.weight_onset = 0` (the default) exactly reproduces the old behavior. Setting it to a positive value (e.g., 0.5) adds a preference for temporally proximate matches, which is useful for long sessions or repetitive event patterns.

The dynamic programming algorithm (both versions)

The DP algorithm itself is identical in both versions — only the similarity function differs.




```
dp = zeros(n+1, m+1);
for i = 1:n
  for j = 1:m
    dp(i+1,j+1) = max([ ...
      dp(i,j)   + similarity_fun(x1(i,:), x2(j,:)), ... % match i to j
      dp(i+1,j),                                      ... % skip j (missing in
          events1)
      dp(i,j+1)  ]);                                      % skip i (missing in
          events2)
  end
end
% Backtrack from dp(n+1, m+1) to recover paired indices
```

Listing 8: DP recurrence (same in old and new versions)

This is identical in structure to the Longest Common Subsequence (LCS) algorithm, but with a continuous similarity score instead of binary match/mismatch. Skipping costs nothing (dp stays flat), while matching adds the similarity score. The backtracking recovers the set of pairs that maximises the total accumulated similarity.

How `bml_sync_digital` uses the DP output

After DP matching, `bml_sync_digital` fits a linear model:

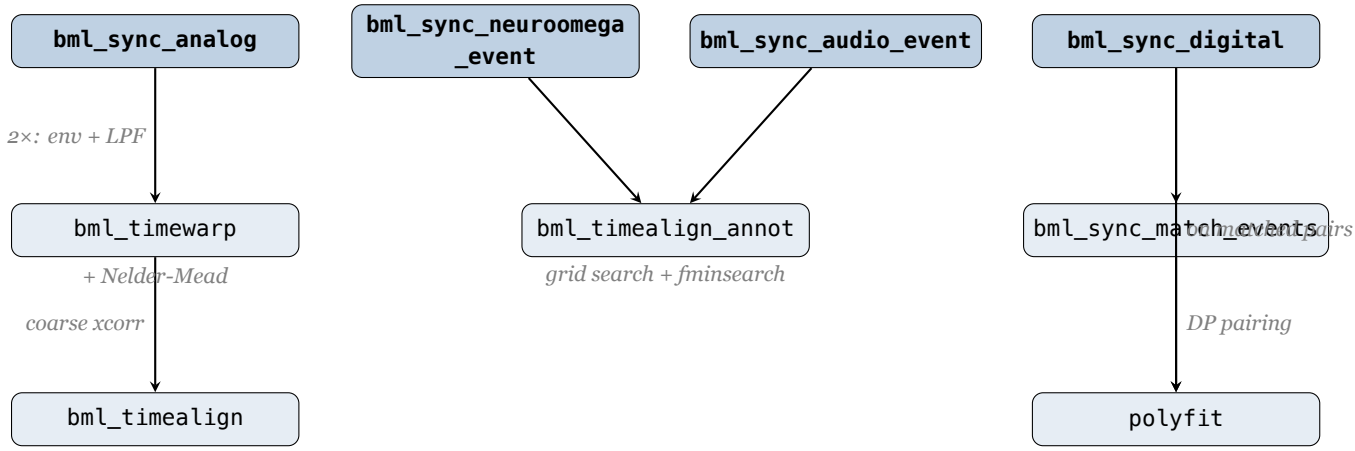
```
% dtv = time differences for all matched pairs
dtv = master_events.starts(idxs_master) - slave_events.starts(idxs_slave);

% Detect contiguous high-similarity chunks
% (similarity drops when event sequences diverge)
cfg1.threshold = sim_threshold;   % default 0.9
chunks = bml_annot_detect(cfg1, sim_raw);  % segments where sim > 0.9

% For each chunk: fit linear model
for i = 1:height(chunks)
  chunk_idxs = ceil(chunks.starts(i)):floor(chunks.ends(i));
  p = polyfit(master_events.starts(idxs_master(chunk_idxs)) - tbar, ...
              dtv(chunk_idxs), 1);
  chunks.warp(i)    = p(1);
  chunks.delta_t(i) = p(2);
end

% Consolidate chunks with consistent delta_t
cfg1.criterion = @(x) (abs(max(x.delta_t) - min(x.delta_t)) < cfg.timetol);
cons_chunks = bml_annot_consolidate(cfg1, chunks);
```

```
21
22 % Final polyfit across all events in each consolidated chunk
23 % If slave has a 'sample' column, also compute (s1,t1,s2,t2) coordinates:
24 p = polyfit(slave_events.sample(idxs(chunk_idxs)), ...
25             master_events.starts(idxs_master(chunk_idxs)), 1);
26 cons_chunks.s1(i) = slave_events.sample(idxs(chunk_idxs(1)));
27 cons_chunks.t1(i) = cons_chunks.s1(i) * p(1) + p(2);
28 cons_chunks.s2(i) = slave_events.sample(idxs(chunk_idxs(end)));
29 cons_chunks.t2(i) = cons_chunks.s2(i) * p(1) + p(2);
```

Alignment Function Hierarchy

This section provides a complete map of the low-level alignment functions, how they relate to each other, and when each one is appropriate.

Call graph



The two dimensions

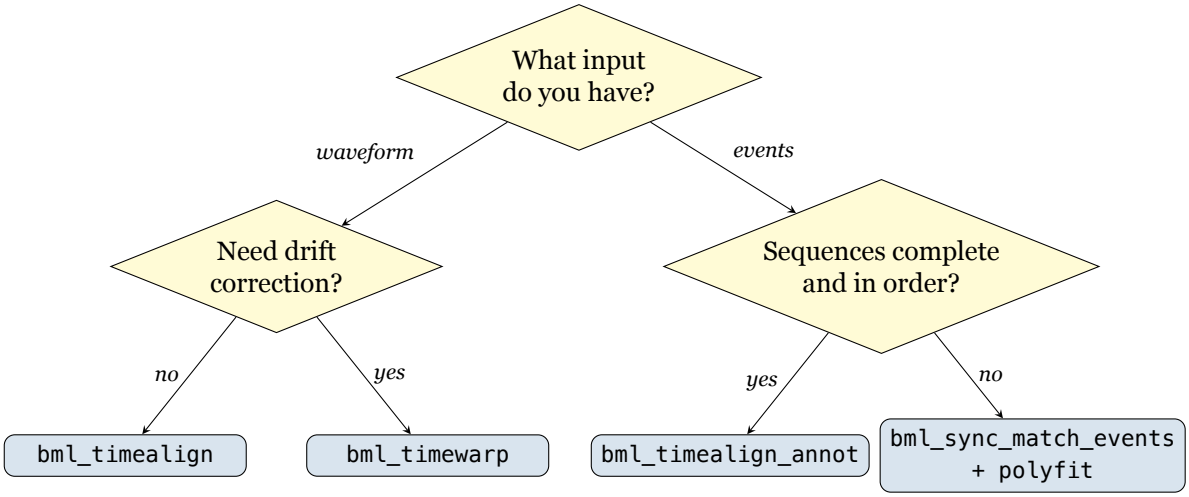
There are two independent choices:

	Waveform input	Event list input
Offset only	`bml_timealign`	`bml_timealign_annot`
Offset + drift	`bml_timewarp`	`bml_timewarp_annot`

- **Waveform**: continuous `FT_DATATYPE_RAW` signal (e.g. shared analog click track)
- **Event list**: annotation table of timestamps (e.g. TTL pulses, audio peaks)
- **Offset**: constant shift δt only
- **Drift**: linear clock rate $ws_1 \neq 1$ fitted in addition to offset

`bml_timewarp_annot` is an alias for `bml_timealign_annot` with `timewarp=true` forced. It is not called by any pipeline function — all callers use `bml_timealign_annot` directly.

Decision guide: which function to call?



Situation	Call this	Why
Zoom analog click track	`bml_sync_analog`	shared waveform, xcorr + warp
Zoom audio peaks (no shared cable)	`bml_sync_audio_event`	peaks ☐ events, then grid search
NeuroOmega TTL pulses	`bml_sync_neuroomega_event`	same signal, grid search
Psychtoolbox task events	`bml_sync_digital`	sequences may differ ☐ DP
Custom: waveform, with drift	`bml_timewarp`	
Custom: events, known complete	`bml_timealign_annot`	
Custom: events, possibly missing	`bml_sync_match_events`	

Why does NeuroOmega use `bml_timealign_annot` instead of DP? Both NeuroOmega and Ripple are wired to the same TTL cable, so every pulse recorded by one is also recorded by the other. The sequences are identical — you only need to find the time shift, not figure out which event matches which. The simpler grid-search approach is sufficient and faster.
Psychtoolbox events, by contrast, travel over a potentially unreliable channel and represent a different set of event types. The DP algorithm in `bml_sync_match_events` is needed to establish correspondence before estimating the offset.

Event Alignment Internals: `bml_timealign_annot`

Used by audio and NeuroOmega sync. Finds δt (and optionally ξ) that minimises the total event mismatch.

$$\mathcal{C}(\delta t) = \sqrt{\sum_i \left[\min\left(\min_j |s_i + \delta t - m_j|,\ t_{\text{clip}}\right)\right]^2}$$

Algorithm: (1) brute-force scan on `linspace(-scan, scan, 2·scan/scan_step+1)`; (2) censor outlier events with residual > `censor_mismatch`; (3) `fminsearch` refinement in 1-D (no warp) or 2-D (with warp).

Continuous Signal Alignment

`bml_timealign` — cross-correlation

1. Compute scan range (max possible shift before files stop overlapping)
2. Pad both signals to cover the scan window; resample to `resample_freq` (10 kHz)
3. Preprocess: **envelope** (abs of Hilbert, downsample to 100 Hz) or **LPF** (4th-order Butterworth)
4. Normalize: subtract robust median, divide by robust std (16th–84th percentile)
5. `xcorr(..., 'coeff')`, find peak lag → `slave_delta_t`
6. If LPF method: also test inverted polarity; keep best

`bml_timewarp` — linear warp optimization

$$w(t) = t_{\text{pivot}} + wt_0 + (t - t_{\text{pivot}}) \cdot ws_1$$

$$\mathcal{L}(wt_0, ws_1) = -\frac{\langle f(w(\mathbf{t})), p\rangle}{\|f(w(\mathbf{t}_0))\|\|p\|} + \left(\frac{wt_0}{P_{\delta t}}\right)^2 + \left(\frac{ws_1 - 1}{P_\xi}\right)^4$$

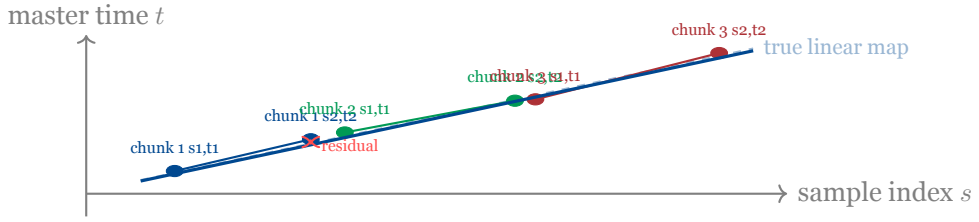
Minimised by Nelder-Mead simplex (`fminsearch`). Output: new (s_1, t_1, s_2, t_2) and `warpfactor` $= 1/ws_1$.

Consolidation: What It Is and Why It Matters

The problem consolidation solves

After running `bml_sync_analog` (or any event sync), the `sync_roi` table has **multiple rows per file** — one per chunk. For example, a 5-minute Zoom recording synced in 3 chunks produces 3 rows, each with slightly different `(s1,t1,s2,t2)` estimates.

Before we can use these coordinates for data loading, we need to answer: do all three estimates agree? And if so, what is the *single* best linear mapping for the whole file?



Each chunk gives two anchor points (a short line segment). If all three segments lie on the same global line (residuals < 1 ms), consolidation succeeds and we replace the three rows with one, using the global best-fit line.

Level 1: per-file consolidation

```
% For a file with N chunks, bml_sync_consolidate does:
% 1. Convert all (s1,s2) to raw (global) sample indices via s2raw()
i_roi = s2raw(i_roi);   % adds columns raw1, raw2

% 2. Fit a single line through all anchor points
[p, max_delta_t, off_idx] = sync_cons_group(i_roi, timewarp, 'raw');
%    p(1) = 1/Fs_eff (slope), p(2) = time intercept
%    max_delta_t = max residual in seconds

% 3. Accept or reject
if max_delta_t <= timetol   % default 1 ms
  i_roi.t1 = polyval(p, i_roi.raw1);
  i_roi.t2 = polyval(p, i_roi.raw2);
  i_roi.warpfactor = 1 ./ (i_roi.Fs .* p(1));
else
  error('consolidation failed: max residual %f ms', max_delta_t*1000);
end
```

Level 2: contiguous file consolidation

For devices that split recordings into consecutive files (Zoom, NeuroOmega), the same logic is applied *across* files of the same type. The criterion for attempting this is that the files are contiguous in time (gap $<$ `timetol_contiguous`):

```
% Detect contiguous file groups
cfg1.criterion = @(x) ...
  (abs(sum(x.duration) - max(x.ends) + min(x.starts)) < height(x)*
      timetol_contiguous);
i_roi_cont = bml_annot_consolidate(cfg1, bml_roi_confluence(i_roi));

% For each contiguous group: fit one global line across all files
% After fitting, set junction times at file boundaries:
for i = 1:(height(i_roi_cont_j)-1)
  % Midpoint between adjacent anchor times = junction
  junction = (i_roi_cont_j.t2(i) + i_roi_cont_j.t1(i+1)) / 2;
  i_roi_cont_j.ends(i)     = junction;
  i_roi_cont_j.starts(i+1) = junction;
end
```

The **midpoint junction rule** ensures no gap and no overlap between adjacent files. The timing uncertainty at each file boundary is split evenly.

What the consolidated table looks like

```
cfg_cons = [];
cfg_cons.roi               = [sync_roi_analog; sync_roi_no; sync_roi_ptb];
cfg_cons.timetol           = 1e-3;
cfg_cons.timetol_contiguous = 1e-3;
cfg_cons.contiguous        = true;
cfg_cons.group             = 'session_id';
```

```
cfg_cons.plot_diagnostic   = true;  % shows pairwise residual matrix per file
sync_roi_final = bml_sync_consolidate(cfg_cons);
```

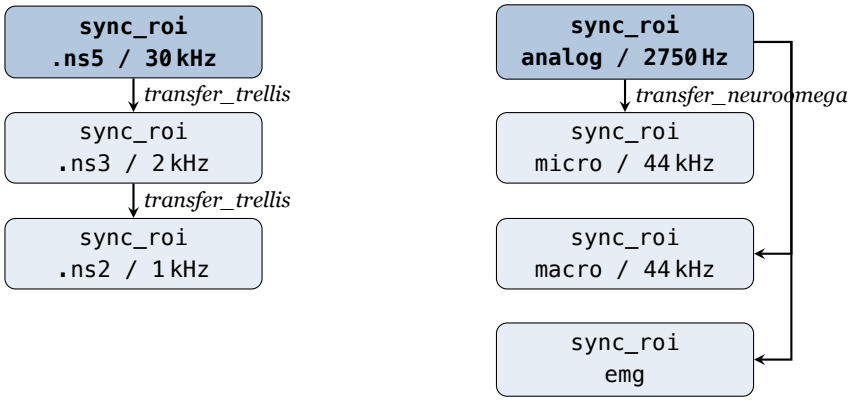
id	starts	ends	s1	t1	s2	t2	name	Fs	warpfactor	filetype
1	52310.0	52610.0	1	52310.000	9000000	52609.999	session01.nev	30000	1.000000	trellis
2	52313.2	52612.9	1	52313.201	13219590	52612.897	zoom_001–3	44100	0.999998	zoom
3	52311.7	52611.1	1	52311.702	6001200	52611.098	NO_001–2	20020	1.000003	neuroomega

Interpreting warpfactor: Zoom at 0.999998 means its clock ran 2 ppm *fast* (slightly fewer samples per real second than 44100). NeuroOmega at 1.000003 means its clock ran 3 ppm *slow*. Over a 5-minute session: Zoom off by 0.6 ms, NeuroOmega off by 0.9 ms — easily within 1 ms tolerance.

Sync Transfer Functions

After synchronization you have `(s1,t1,s2,t2)` coordinates for the *file you actually synced* — e.g. the Trellis `.ns5` file or the NeuroOmega `analog` channel. But the same session also contains files at different sampling rates (`.ns2`, `.ns3`) and other channel types (`micro`, `macro`, `emg`) that were never directly synced.

The three transfer functions **propagate** the sync coordinates to all those files without re-running sync.



`bml_sync_transfer_trellis_filetype` — rescale sample indices

Trellis writes the same session as multiple NSx files simultaneously. You sync on `.ns5` (30 kHz, best waveform quality), then transfer to `.ns2` (1 kHz LFP) and `.ns3` (2 kHz).

The function reconstructs the absolute file start/end times t_0, t_f from the synced coordinates, then maps sample indices by the nominal Fs ratio:

$$s_1^{\text{new}} = \left\lceil \frac{F_s^{\text{new}}}{F_s} \cdot \left(s_1 - \tfrac{1}{2}\right) \right\rceil, \qquad t_1^{\text{new}} = t_0 + \frac{s_1^{\text{new}} - \frac{1}{2}}{F_{s,\text{eff}}^{\text{new}}} - \text{delay}$$

The `delay` parameter corrects for online low-pass filter phase delays (relevant for `.ns2`).

```
cfg = [];
cfg.roi           = sync_ns5;
cfg.filetype      = 'trellis.ns2';
cfg.sync_filetype = 'trellis.ns5';
cfg.delay         = 0;          % optional: correct LPF phase delay (s)
```

```
sync_ns2 = bml_sync_transfer_trellis_filetype(cfg);
```

> **`bml_sync_transfer_nsx_filetype`** is functionally identical but uses `round(…,'significant')` for the real-Fs calculation. Use it when `bml_sync_transfer_trellis_filetype` produces spurious coordinate values with certain NSx files.

`bml_sync_transfer_neuroomega_chantype` — re-anchor to channel internal clock

A single NeuroOmega `.mat` file stores every channel type with its own internal clock: `ChannelName_TimeBegin` and `ChannelName_TimeEnd`. Sync is done on `analog` (2750 Hz). This function re-anchors the coordinates to `micro`, `macro`, `emg`, etc.

chantype	Example channel	Fs
`analog`	`CANALOG_IN_1`	2750 Hz
`micro`	`CRAW_01___Central`	44 kHz
`micro_hp`	`CSPK_01___Central`	44 kHz
`micro_lfp`	`CLFP_01___Central`	~1.4 kHz
`macro`	`CMacro_RAW_01`	44 kHz
`macro_lfp`	`CMacro_LFP_01`	~1.4 kHz
`emg`	`CEMG_1___01`	variable

```
cfg = [];
cfg.roi      = sync_ao_analog;   % synced with chantype='analog'
cfg.chantype = 'micro';
sync_ao_micro = bml_sync_transfer_neuroomega_chantype(cfg);

cfg.chantype = 'macro';
sync_ao_macro = bml_sync_transfer_neuroomega_chantype(cfg);
```

Why not re-sync each file type separately?

- The `analog` channel is used for NeuroOmega sync because its lower Fs (2750 Hz) makes the grid search faster and more robust.
- The `.ns5` file is used for Trellis sync because it has the highest Fs and the cleanest waveform of the click track.
- Re-syncing each file independently introduces independent estimation errors. Transfer ensures all channel types share the *exact same* sync estimate, maintaining perfect internal consistency.

Putting it all together

```
% 1. Sync Trellis ns5
sync_ns5 = bml_sync_analog(cfg_analog);

% 2. Transfer to other Trellis filetypes
sync_ns2 = bml_sync_transfer_trellis_filetype(...
```

```
    struct('roi',sync_ns5,'filetype','trellis.ns2','sync_filetype','trellis.ns5

% 3. Sync NeuroOmega (analog channel)
sync_ao_analog = bml_sync_neuroomega_event(cfg_ao);

% 4. Transfer to micro and macro
sync_ao_micro = bml_sync_transfer_neuroomega_chantype(...
    struct('roi',sync_ao_analog,'chantype','micro'));
sync_ao_macro = bml_sync_transfer_neuroomega_chantype(...
    struct('roi',sync_ao_analog,'chantype','macro'));

% 5. Consolidate everything
sync_roi_final = bml_sync_consolidate(struct('roi', ...
    [sync_ns5; sync_ns2; sync_ao_analog; sync_ao_micro; sync_ao_macro]));
```

Complete End-to-End Example

```
%% STEP 1 -- Build initial ROI
roi = bml_roi_table(bml_info_raw(struct('folder','/data/p001/s01')));

%% STEP 2 -- Define chunks
session = bml_annot_table(table(52310, 52610, 'VariableNames',{'starts','ends'}))
    ;
session.session_id = 1;
chunks = bml_chunk_sessions(session, 3);   % 3 chunks of ~100 s

%% STEP 3 -- Analog sync: Trellis + Zoom
sync_ch = table({'trellis';'zoom'},{'ainp1';'Ch1'},{'analog';'analog'}, ...
          'VariableNames',{'filetype','channel','chantype'});
sync_ch.threshold = [NaN; NaN];

cfg_a = [];
cfg_a.roi = roi(ismember(roi.filetype,{'trellis','zoom'}),:);
cfg_a.chunks = chunks;  cfg_a.master_filetype = 'trellis';
cfg_a.sync_channels = sync_ch;
cfg_a.timewarp = true;  cfg_a.env_scan = 300;  cfg_a.lpf_scan = 1;
sync_roi_az = bml_sync_analog(cfg_a);

%% STEP 4 -- Extract master events from Trellis
master_events = bml_event2annot([], bml_read_event(roi(strcmp(roi.filetype,'
    trellis'),:)));
stim_events = master_events(strcmp(master_events.type,'STIM_ON'),:);

%% STEP 5 -- Event sync: NeuroOmega
cfg_no = [];
cfg_no.roi = roi(strcmp(roi.filetype,'neuroomega'),:);
cfg_no.master_events = stim_events;
cfg_no.scan = 100;  cfg_no.coarsetol = 100;  cfg_no.timewarp = false;
sync_roi_no = bml_sync_neuroomega_event(cfg_no);

%% STEP 6 -- Event sync: Psychtoolbox
ptb = bml_annot_read_tsv('sub-p001_ses-intraop_task-speech_events.tsv');
ptb_triggers = ptb(strcmp(ptb.trial_type,'STIM_ON'),:);
ptb_triggers.value = ones(height(ptb_triggers),1);

```

```
trellis_triggers = stim_events;
trellis_triggers.value = ones(height(trellis_triggers),1);

cfg_ptb = [];
cfg_ptb.timetol = 1e-3;  cfg_ptb.sim_threshold = 0.9;
cfg_ptb.weight_onset = 0.2;  cfg_ptb.onsettol = 200;
cfg_ptb.diagnostic_plot = true;
[sync_roi_ptb, hF] = bml_sync_digital(cfg_ptb, trellis_triggers, ptb_triggers);

%% STEP 7 -- Consolidate everything
cfg_cons = [];
cfg_cons.roi = [sync_roi_az; sync_roi_no];
cfg_cons.timetol = 1e-3;  cfg_cons.contiguous = true;  cfg_cons.group = '
    session_id';
sync_roi_final = bml_sync_consolidate(cfg_cons);
save('/data/p001/s01/sync_roi.mat','sync_roi_final');

%% STEP 8 -- Apply PTB sync and use synchronized data
delta_t = sync_roi_ptb.delta_t(1);
warpfactor = sync_roi_ptb.warpfactor(1);
tbar = mean(ptb_triggers.starts);
ptb.starts = (ptb.starts - tbar) .* warpfactor + tbar + delta_t;
ptb.ends   = (ptb.ends   - tbar) .* warpfactor + tbar + delta_t;
% All ptb event times are now in Trellis master clock

% Load Zoom audio for a specific time window (in master time)
cfg_load = [];
cfg_load.roi = sync_roi_final(strcmp(sync_roi_final.filetype,'zoom'),:);
cfg_load.toi = [52350, 52400];   % master clock seconds
zoom_raw = bml_load_continuous(cfg_load);
```

Listing 9: Complete synchronization script: Trellis + Zoom + NeuroOmega + Psychtoolbox

Common Pitfalls

1. **id is not stable.** Reassigned at every `bml_annot_table()` call. Use as a within-table label only.
2. **The $\pm 0.5/F_s$ midpoint offset.** `t1 = starts + 0.5/Fs`, not `starts`. Forgetting this causes systematic half-sample errors.
3. **$F_{s,\text{eff}} \neq F_s$ after warping.** Always use `bml_idx2time`; never divide by `Fs` directly.
4. **chunk_extend too small.** Set `cfg.chunk_extend` $\geq$ expected OS timestamp error (5–30 s typical). If too small, the slave signal will not overlap the master in the extended chunk.
5. **coarsetol too small (NeuroOmega/PTB).** If OS timestamps are off by more than `coarsetol`, the correct master events are excluded from the alignment window. Increase `coarsetol` to match the expected offset.
6. **DP returns > 10 chunks.** `bml_sync_digital` raises an error if similarity drops below `sim_threshold` more than 10 times. This usually indicates poor event matching — try adjusting weights or lowering `sim_threshold`.
7. **Consolidation failure.** A bad chunk (one where sync was unreliable) will cause the per-file polyfit to have large residuals. Use `cfg.plot_diagnostic = true` to identify it

and re-run the sync for that chunk only.

8. **Combining analog and event sync ROI.** `bml_sync_analog` adds `chunk_id` and `sync_channel` columns; event sync functions may not. Make sure schemas match before vertically concatenating.

9. **Warpfactor $\gg 1$ or $\ll 1$.** Values deviating from 1 by more than 10^{-4} (100 ppm) are suspicious. Check whether the sync signal was clipped, noisy, or whether the wrong channel was selected.

Diagnostic Plots: How to Enable and Interpret

Every sync function exposes a diagnostic flag. Enable it during development and whenever a sync fails validation.

```
cfg.diagnostic_plot = true;     % bml_sync_analog, bml_sync_neuroomega_event,
                                % bml_sync_audio_event, bml_sync_digital,
                                % bml_sync_match_events
cfg.plot_diagnostic = true;     % bml_sync_consolidate only
```

`bml_sync_match_events` — DP similarity plot

Produces a 2-panel figure.

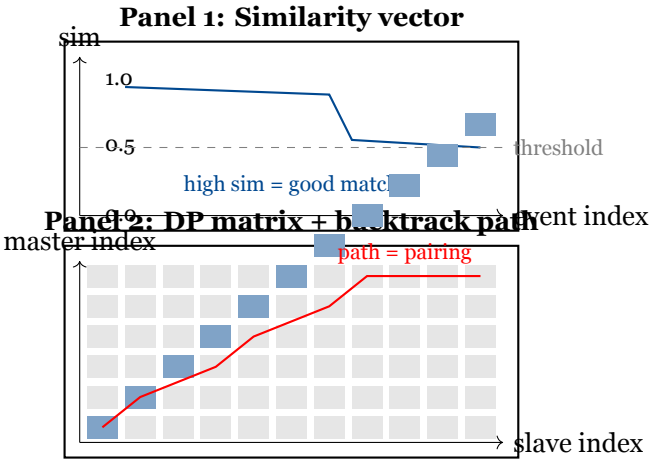


Figure 1: Schematic of `bml_sync_match_events` diagnostic figure.

Pattern	Meaning
All similarity values near 1	Perfect match
Cluster of high values then drop	One clean matched block; use `sim_threshold` to isolate it
All values < 0.5	Poor match — wrong scan range or no overlap
Backtrack path is a smooth diagonal	Clean 1:1 correspondence
Path has horizontal/vertical runs	Events missing on one side
Path stays in one corner	No real overlap — wrong files

`bml_sync_audio_event` / `bml_sync_neuroomega_event` — event alignment plot

One 2-panel figure per file: a timeline (left) and a residual histogram (right).

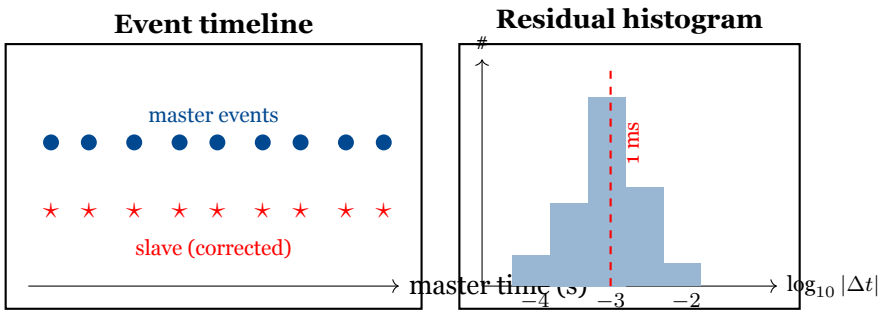


Figure 2: Schematic of event-alignment diagnostic. Good output: circles and stars overlap; histogram peak at $\log_{10}|\Delta t| \leq -3$ (i.e. ≤ 1 ms).

Pattern	Meaning
Circles and stars overlap	Good alignment
Stars consistently offset	`delta_t` estimate slightly off; widen scan
Stars spread out	Residual drift; enable `timewarp = true`
Many extra stars	False peaks; increase `min_rph`
Many circles without a star	Missed events; decrease `min_rph`
Histogram peak at ≤ -3 ($\leq$1 ms)	Excellent
Peak at -2 (10 ms)	Marginal; borderline for neural data
Peak at -1 or higher	Poor; inspect timeline
Bimodal histogram	Matched and unmatched events mixed

`bml_sync_digital` — residual + alignment plot

One 3-panel figure per contiguous matched chunk.

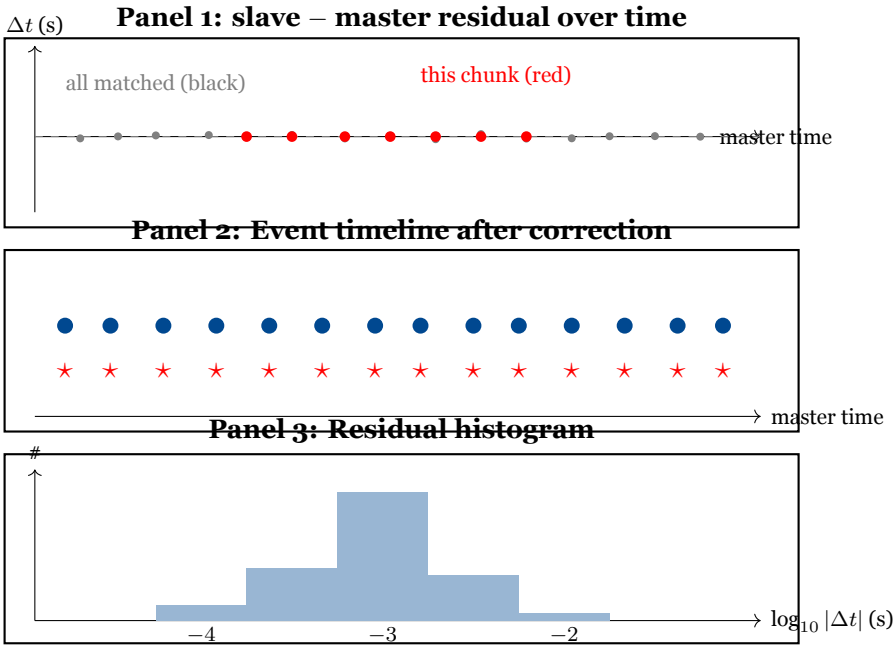


Figure 3: Schematic of `bml_sync_digital` 3-panel diagnostic. Good output: Panel 1 residuals are flat near 0; Panel 2 circles and stars overlap; Panel 3 peak at ≤ -3.

Panel 1 interpretation:

Pattern	Meaning
Flat horizontal band near 0	Pure offset, no drift — good
Tilted straight line	Linear drift; enable `timewarp = true`
Curved or scattered	Non-linear or bad matching
Sudden jump	Two sessions mixed; check `cfg.group`

`bml_sync_consolidate` — pairwise residual heatmap

One $N \times N$ heatmap per file (Level 1) and per contiguous group (Level 2), where N = number of chunks. Color = pairwise δt disagreement.

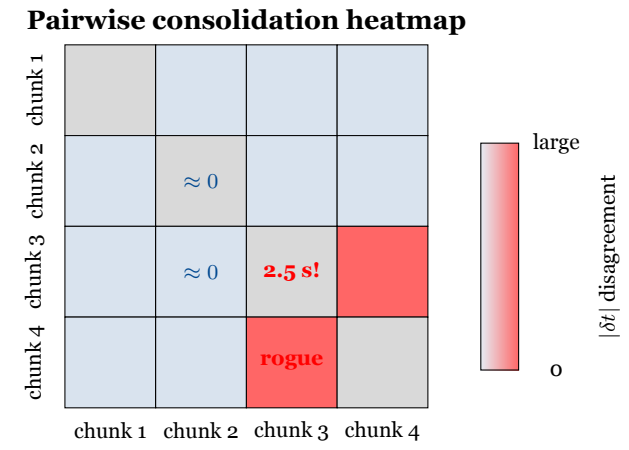


Figure 4: Schematic consolidation heatmap. Blue = good agreement; red = rogue chunk. Remove the row/column that is uniformly bright and re-run consolidation.

Pattern	Meaning
All cells dark	All chunks agree — consolidation will succeed
One row/column bright	That chunk is a rogue; remove and retry
Checkerboard	Alternating drift; shorten chunk windows
All cells bright	Sync failed entirely

Listing 10: Removing a rogue chunk and re-consolidating

```
% Find rogue chunks (>5 ms residual)
bad = sync_roi.meanerror > 0.005;
sync_roi_clean = sync_roi(~bad, :);
sync_roi_final = bml_sync_consolidate(struct('roi', sync_roi_clean));
```

Quick-reference: what to check first

Question	Plot	Panel
Did DP find the right event pairs?	`bml_sync_match_events`	DP matrix path
Are residuals < 1 ms?	Any function	Residual histogram
Is there drift?	`bml_sync_digital`	Residual over time
Is one chunk an outlier?	`bml_sync_consolidate`	Pairwise heatmap
Are all events accounted for?	Audio/NeuroOmega	Timeline
Is similarity high enough?	`bml_sync_match_events`	Similarity vector

Best Practices and Lab Conventions

Choose your sync method by signal quality

Situation	Method	Why
Zoom + shared analog waveform on Trellis	Approach A: `bml_sync_analog`	Exploits full waveform shape; most robust
Zoom + only TTL click track	Approach B: `bml_sync_audio_event`	Audio peaks matched to Ripple digital events
NeuroOmega + shared TTL	`bml_sync_neuroomega_event`	TTL events present on both devices
Psychtoolbox / task laptop	DP matching + fallback	Clock offset can be hours; DP handles missing events
Session > 30 min	Enable `timewarp=true`	Drift accumulates; linear warp prevents error growth

Always validate sync quality

Before saving or using `sync_roi`, check:

```
% For analog sync: warpfactor should be very close to 1
assert(all(abs(sync_roi.warpfactor - 1) < 100e-6), ...
  'warpfactor deviation > 100 ppm -- check sync channel');

% For event sync: mean_sim should be high
assert(all(sync_roi.mean_sim > 0.8), ...
  'low similarity -- check event matching');

% After consolidation: residuals < 1 ms
cfg_check.timetol = 1e-3;
bml_sync_check(cfg_check, sync_roi_final);
```

Annotation tables as the lingua franca

Every data structure in BML is an annotation table. This is intentional:

- Use `bml_annot_table()` to create any new table — it enforces `id`, `starts`, `ends`, `duration` and checks for overlaps.
- Never store times as arrays. Always put them in a table with `starts`/`ends` so they can be filtered, joined, and validated.
- `bml_annot_filter(A, B)` is the workhorse for restricting one table to the time range of another. Use it before any alignment to avoid processing irrelevant events.

Save intermediate outputs

Sync is computationally expensive and hard to reproduce if session conditions change. Save:

```
save(fullfile(PATH_SYNC, 'sync_roi_final.mat'), 'sync_roi_final');
writetable(sync_roi_final, fullfile(PATH_SYNC, 'sync_roi_final.tsv'), ...
  'FileType','text','Delimiter','\t');
```

Save the *un-consolidated* `sync_roi` from each method too (analog, neuroomega, ptb). If consolidation later fails for one file, you can re-run only that step.

The fallback-first principle

When applying sync to event files, always prefer exact matches over interpolation (see Section 8.6). The priority order is:

1. **Exact Ripple timestamp** (matched via DP): hardware-precision ground truth.
2. **Linear warp** from the best DP-matched contiguous block: corrects offset + drift, accurate to ∼1–2 ms.
3. **No sync** (NaN): flag events that fall outside the synced session window rather than extrapolating.

BIDS event file conventions

All task event logs should follow BIDS:

- Columns: `onset` (seconds from task start), `duration`, `trial_type`, `response_time` (optional)
- After sync: save as `*_sync.tsv` with `starts` (master clock seconds) replacing `onset`
- `bml_annot_read_tsv` handles the `onset`→`starts` rename automatically

Chunking strategy

- Chunk duration of 60–100 s is a good default. Shorter chunks have less data for correlation (noisier estimates); longer chunks miss local drift changes.
- For long sessions (>2 h), prefer 100 s chunks with `timewarp=true`.
- `chunk_extend` (extra seconds to load beyond the chunk boundary) should be at least as large as your expected OS timestamp error (typically 5–30 s).

Toward Python: design principles to carry forward

The BML toolbox is currently implemented in MATLAB. The long-term goal is to migrate to Python (NumPy/Pandas/MNE). The following principles in the current design map naturally to Python:

MATLAB (BML)	Python equivalent	Key invariants
Annotation `table`	`pandas.DataFrame`	Always has `starts`, `ends`, `duration` columns
ROI table	`DataFrame` + `SyncCoords` dataclass	`s1,t1,s2,t2` define the linear map
`bml_idx2time`	Pure function → `numpy` vectorized	No side effects; invertible
`bml_annot_filter`	`df[df.starts >= t0 & ...]`	Interval overlap logic unchanged
`bml_annot_consolidate`	`groupby` + merge	Criterion function → lambda
DP matching	`scipy` or pure Python	Identical recurrence; only similarity function changes
`FT_DATATYPE_RAW`	`mne.io.RawArray`	Single trial, single channel for sync functions

When porting, preserve the two-coordinate system `(s1,t1,s2,t2)`: it is the core abstraction that decouples sample indices from master-clock times, and nothing in the Python ecosystem provides an equivalent out of the box.

Mathematical Reference

Index-to-time and inverse

$$F_{s,\text{eff}} = \frac{s_2 - s_1}{t_2 - t_1}, \qquad t(i) = \frac{i}{F_{s,\text{eff}}} - \frac{0.5}{F_{s,\text{eff}}} + \frac{s_2 t_1 - t_2 s_1}{s_2 - s_1}$$

$$i(t) = \text{round}\left(\frac{t_2 s_1 - s_2 t_1 + (s_2 - s_1)\, t}{t_2 - t_1} \right)$$

Time-warp model and cost

$$w(t) = t_{\text{pivot}} + wt_0 + (t - t_{\text{pivot}}) \cdot ws_1$$

$$\mathcal{L}(wt_0, ws_1) = -\frac{\langle f(w(\mathbf{t})), p \rangle}{\|f(w(\mathbf{t}_0))\|\, \|p\|} + \left(\frac{wt_0}{P_{\delta t}} \right)^2 + \left(\frac{ws_1 - 1}{P_\xi} \right)^4$$

$$t_1^{\text{new}} = t_{\text{pivot}} - wt_0 - \frac{t_2^{\text{crop}} - t_1^{\text{crop}}}{2\, ws_1}, \qquad t_2^{\text{new}} = t_{\text{pivot}} - wt_0 + \frac{t_2^{\text{crop}} - t_1^{\text{crop}}}{2\, ws_1}$$

Event alignment cost (`bml_timealign_annot`)

$$\mathcal{C}(\delta t) = \sqrt{\sum_i \left[\min\left(\min_j |s_i + \delta t - m_j|,\ t_{\text{clip}} \right) \right]^2}$$

Event similarity (new 5-feature, `bml_sync_match_events`)

$$\text{sim}(\mathbf{x}_i, \mathbf{x}_j) = \frac{1}{W} \left[w_1 \cdot \frac{1}{1 + (\Delta t^{\text{pre}}/\tau)^2} + w_2 \cdot \frac{1}{1 + (\Delta t^{\text{post}}/\tau)^2} + w_3 \cdot \mathbb{1}[v_1^{\text{pre}} = v_2^{\text{pre}}] + w_4 \cdot \mathbb{1}[v_1^{\text{post}} = v_2^{\text{post}}] + w_5 \right.$$

DP recurrence

$$\mathrm{dp}[i{+}1, j{+}1] = \max(\mathrm{dp}[i, j] + \mathrm{sim}(\mathbf{x}_i, \mathbf{x}_j),\ \mathrm{dp}[i{+}1, j],\ \mathrm{dp}[i, j{+}1])$$